Unit testy a práca s pamäťou - doplnok

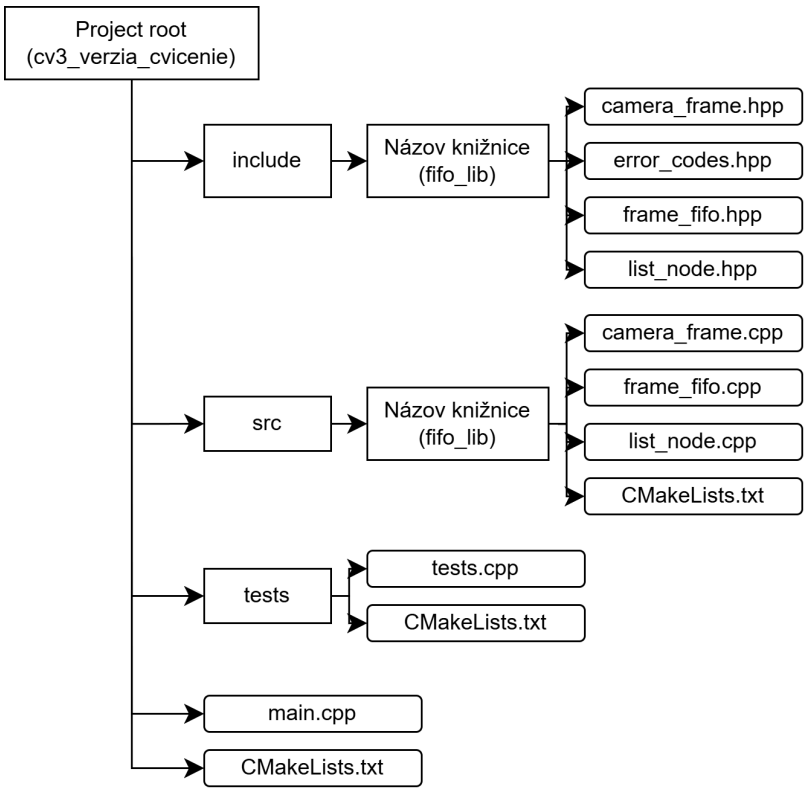
Pre lepšie pochopenie dnešného cvičenia uvádzame tento doplnkový materiál s preberanými témami.

Prehľad kódu cv3_verzia_cvicenie

Kód obsahuje zjednodušenú implementáciu FIFO kontajnera na štýl std::queue<> pre objekty CameraFrame - simulovaných dát z kamery. Do kontajneru možno vložiť prvok na koniec zoznamu pomocou metódy_ pushBack()_ a odstrániť prvok na začiatku zoznamu pomocou metódy popFront(). K akémukoľvek prvku zoznamu je možné pritúpiť metódou at(). CameraFrame obsahuje buffer s dátami, jednoduché vlastné premenné a základné gettre. Po vložení CameraFrame objektu do kontajnera sa stáva read-only, nie je možné jeho prvky ďalej modifikovať.

> main.cpp súbor obsahuje vzorovú logiku na použitie knižnice. V objektovo-orientovanom programovaní by sme by-the-book aj toto mali enkapsulovať do objektov.

Štruktúra projektu so statickou knižnicou



Kompilácia projektu

V priečinku *root* sa nachádza main.cpp súbor s nadradeným CMakeLists.txt. Tento bsahuje typický boilerplate ako

```
cmake_minimum_required(VERSION 3.10)
project(FifoApp VERSION 1.0.0 LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED True)
```

a kompilačné flagy

```
target_compile_options(fifo_app PRIVATE
                       -Wall; -Wextra; -Werror)

target_compile_options(fifo_app PRIVATE
    $<$<CONFIG:Debug>:-fsanitize=undefined>)
target_link_options(fifo_app PRIVATE
    $<$<CONFIG:Debug>:-fsanitize=undefined>)
```

> Na minulom cvičení sme vám ukázali všetky možné -W nastavenia. V tomto sme to zjednodužili na pár najdôležitejších: -Wall, -Wextra a -Werror.

Podradené CMakeLists.txt pre testy a statickú knižnicu už takéto riadky **neobsahujú**.

Aby nadradený CMakeLists.txt vedel podradené CMakeLists.txt zavolať, pridávame priečinky s nimi pomocu *add_subdirectory()*

```
add_subdirectory(src/fifo_lib)
add_subdirectory(tests)
```

Include knižnice

Pri include-ovaní hlavičkových súborov z nami vytvorenej statickej knižnice platia nasledovné zásady:

- Pri *target_include_directories()* zadávame celý priečinok *include*, nie jednotlivé podpriečinky
- Pri *#include* dávame cestu relatívne k priečinku include, t.j

```
#include "nazovpodpriecinku/nazovsuboru.hpp"
```

- Hlavičkový súbor dávame do úvodzoviek namiesto diamantových zátvoriek.

> V tomto prípade použitie úvodzoviek určuje nami zvolený coding style. Reálne je rozdiel medzi nimi nasledovný: Úvodzovky najprv vyvolavú hľadanie súboru v súčasnom priečinku, a až následne v include path. Diamantové zátvorky vyvolajú hľadanie rovno v include path.

Smart pointre/smerníky

Štandardný raw pointer nerieši vlastníctvo a životnosť objektu. Musíme ručne riešiť alokáciu a delokáciu pamäte, čo môže viesť k únikom pamäte, dvojitému uvoľneniu, segfault pri prístupe k nealokovanej pamäti ...

V modernom C++ používame RAII (Resource Acquisition Is Initialization) princíp: objekt má vlastníka a jeho životnosť je prepojená s životnosťou vlastníka. Takýmto RAII vlastníkom sú smart pointre - pri svojom vzniku volajú konštruktor objektu a pri svojom zániku jeho deštruktor.

Rozpoznávame tri typy smart pointrov

Unique pointer

Unique pointer má presne jednu inštanciu. Nemôžeme ho kopírovať, iba presúvať pomocou std::move(). Pri ukončení scopu v ktorom existuje, zaniká.

```
std::unique_ptr a = std::make_unique<Obj>(argumenty konštruktoru);

std::unique_ptr b = std::move(a); // vlastníctvo objektu prechádza na b
```

Shared pointer

Shared pointer môže mať viacero inštancií, ktorých počet drží v use_count(). Dokým je use_count nenulový, objekt žije. Pri zaniknutí shared pointru sa dekrementuje use_count.

```
std::shared_ptr a = std::make_shared<Obj>(argumenty konštruktoru);
std::shared_ptr b = a; // môžem spraviť, use_count() je teraz 2
a.reset() // objekt stále žije, use_count() je teraz 1
b.reset() // objekt zaniká, use_count() je teraz 0
```

Weak pointer

Weak pointer je nevlastniaci pointer na objekt, ktorý je spravovaný shared pointrom. Nezvyšuje jeho use_count(). Pristupujeme k objektu pomocou metódy lock().

```
std::shared_ptr a = std::make_shared<Obj>(argumenty konštruktoru)
std::weak_ptr b(a); //vytváram weak_ptr z a, a.use_count() je stále 1
{ //vytváram scope pre b_temp
std::shared_ptr b_temp = b.lock(); // pristupujem k a cez weak_ptr b,
a.use_count() sa zvýši na 2
}
//b_temp zanikol, a.use_count() je opäť 1
```

Raw pointre/smerníky v C++

Raw pointre v C++ koexistujú so smart pointrami. Vo všeobecnosti by sme mali používať smart pointre všade, kde riešime vlastníctvo - t.j. životný cyklus objektu. Raw pointre používame najmä v prípadoch:

- Pri interakcii so staršími API z jazyka C
- Pri nevlastniacom *nahliadnutí* do objektu
- Pri práci s pamäťovými buffermi (v najnov šších štandardoch nahradené std::span)
- Ako dočasná referencia v rýchlej slučke – objekt sa spravuje smart pointerom alebo kontajnerom, raw pointer sa používa len ako optimalizácia prístupu.

> Raw pointer vieme získať zo smart pointera zavolaným metódy get()

Nullptr keyword

V C sa prádny pointer označoval keywordom *NULL*. V C++ bol pridaný nový keyword *nullptr* špecificky pre pointre, zatiaľ čo *NULL* je použitý ako prázdna hodnota. Používame:

```
uint8_t* ptr = nullptr;
```

V prípade smart pointrov vytvárame prádzny pointer pomocou {}. Jeho hodnota je v takom prípade rovná *nullptr*.

```
std::shared_ptr<int> b = {};
//b == nullptr
```

Alokácia bufferu v modernom C++

V C++ rozpoznávame tri spôsoby alokácie súvislej pamäte na heap:

```
ptr = (uint8_t*)std::malloc(length*sizeof(uint8_t)) a std::free(ptr) // z jazyka
C
ptr = new uint8_t[length] a delete[] ptr // C++, zastarané
ptr = std::make_unique<uint8_t[]>(length) // C++, moderné
```

Z toho:

- malloc je prenesené z jazyka C, a v súčasnosti sa používa už len pre spätnú kompatibilitu so staršími kódmi
- new a delete boli zavedené v C++98, v modernom C++ už boli nahradené alokáciou cez smart pointre
- alokácia pomocou smart pointrov - std::make_unique a std::make_shared - bola zavedená v C++14 a je súčasným štandardom. Jej výhodou je absencia príkazu pre dealokáciu - deje sa automaticky pri zaniknutí smart pointra.

Move konštruktor a std::move

std::move slúži na presun vlastníctva zdrojov (pamäť, handle, mutex ...) objektu. Objekt pretypuje na rvalue (&&) referenciu a umožní volať move konštruktor alebo move assignment, ktoré presunú zdroje do nového objektu.

```
std::unique_ptr<int> a = std::make_unique<int>(7);
std::unique_ptr<int> b = std::move(a); // a už nevlastní nič, všetko vlastní b. a
je po presnuní rovné nullptr
```

Copy konštruktor

Copy konštruktor sa volá, keď vzniká nový objekt ako kópia existujúceho. berie ako vstupný argument const referefenciu na pôvodný objekt, čím zaručuje že pôvodný objekt sa pri kopírovaní nemení. Nový objekt je duplikátom - všetky hodnoty zdrojov má rovnaké, ale je nezávislý a okupuje iné miesto v pamäti.

```
std::string a = "abc";
std::string b = a; //b vznikne pomocou copy konštruktora std::string s const &a
na vstupe. a zostáva platný a nezávislý od b.
```

Const keyword

Const je keyword, ktorý označuje nemennú hodnotu.

```
//V premenných
const int a = 10; //po inicializácii nemôžem meniť hodnotu
const int* p = ptr; //nemôžem meniť hodnotu na ktorú ukazuje pointer
int* const p = ptr; //nemôžem meniť adresu pointra
const int& p = ref; //nemôžem meniť hodnotu premennej, na ktorú ukazuje
referencia
int getVar() const; //funkcia nemení stav objektu, ktorému patrí
const int getVar(); //návratová hodnota funkcie je po vzniknutí nemenná
const std::unique_ptr<Obj> //môžem meniť objekt, ale nemôžem meniť pointer -
nemôžem resetovať ani volať std::move
std::unique_ptr<const Obj> //nemôžem meniť objekt, na ktorý smart pointer ukazuje
```

Error kódy

V mnohých API jazyka C sa používajú error kódy na vyčítanie chyby. V hlavičkovom súbore sa definuje *enum*, kde každému chybovému kódu prislúcha čísaelná hodnota.

```
enum cudaError
{
    cudaSuccess = 0,
    cudaErrorInvalidValue = 1,
    cudaErrorMemoryAllocation = 2,
    cudaErrorInitializationError = 3,
    ...
}
```

V C++ boli pridané výnimky, ktoré v súčasnosti koexistujú s error kódmi. (Budú prebrané na budúcej prednáške).

Výhodou error kódov je malý overhead a ľahká obsluha. Používame pri bežných chybách, ktoré v istej miere očakávame alebo sa dejú často. Výminky sú skôr určené pre významné, neočakávané chyby, ktoré vážne narúšajú chod programu.

Friend testovacia štruktúra

V C++ môžeme pre danú class-u deklarovať *friend* classu/štruktúru/metódu, ktorá následne v rámci svojho scopu môže pristupovať ku všetkým atribútom a metódam danej class-y. Toto nachádza využitie pri white-box testovaní.

```
class FrameFifo //Máme class FrameFifo s private atribútmi
{
   //...
private:
    friend struct FrameFifoTester; //deklarujeme struct FrameFifoTester ako
friend
    int size_ = 0;
};
...

struct  FrameFifoTester
{
    FrameFifoTester(const FrameFifo& fifo) : fifo_(fifo) {} //Konštruktor berie
referenciu na FrameFifo, inicializuje fifo_.

    const int& getSize() const //Friend prístup platí iba v rámci štruktúry,
preto potrebujeme spraviť prístupové metódy
    {
        return fifo_.size_; //size_ je private, ale v rámci tejto friend
štruktúry môžem pristúpiť
    }

private:
    const FrameFifo& fifo_; //Uchovávame si referenciu na FrameFifo. Plánujeme
iba nahliadať stav, takže dávame const.
};
```

Friend test

Google test definuje macro *FRIEND_TEST*, ktoré umožní zaregistrovať konkrétny test ako *friend* danej class-y.

```
#include <gtest/gtest.h>

class CameraFrame
{
private:
    int atribut1=0;
    int atribut2=0;

    FRIEND_TEST(CameraFrameTest, Atribut1Je0); //Zaregistrujem test v definícii
class-y
};

...

TEST(CameraFrameTest, Atribut1Je0)
{
    CameraFrame a;
    EXPECT_EQ(a.atribut1,0); //Môžem spraviť, test je friend CameraFrame
```

```
}
```

V praxi ide o jednoduchú metódu ako spraviť white-box test, bez vytvárania dodatočných štruktúr. Musíme ale zaregistrovať každý test zvlášť, čo v prípade veľkého množstva testov rýchlo spraví kód neprehľadným. Taktiež narúša oddelenie testov a statickej knižnice.